Logit Model (Elastic Net)

James Pease, Natalie Mukoda, Angela Flores Figueroa

2024-11-22

Loading data

```
install.packages(c('readr', 'glmnet', 'foreach', 'parallel'))
```

```
## package 'readr' successfully unpacked and MD5 sums checked
## package 'glmnet' successfully unpacked and MD5 sums checked
## package 'foreach' successfully unpacked and MD5 sums checked
##
## The downloaded binary packages are in
##  C:\Users\Natal\AppData\Local\Temp\RtmpK6IhJd\downloaded_packages
```

```
library(readr)

setwd("C:\\Users\\Natal\\OneDrive\\Documents\\MSBA207-Machine Learning")

train_X <- read_csv("train_X_logit.csv")
train_y <- read_csv("train_y_logit.csv")

valid_X <- read_csv("valid_X_logit.csv")
valid_y <- read_csv("valid_y_logit.csv")
```

Creating model using Lasso

```
library(glmnet)
set.seed(123)

# Convert y to a vector
y <- as.vector(train_y[[1]]) # Extract the first column as a vector

# creating a matrix with predictors
X <- model.matrix(~ ., data = train_X)[, -1] #remove intercept as glmnet will add intercept

cv.lasso <- cv.glmnet(X, y, alpha = 1, family = "binomial")
# Fit the final model on the training data
model <- glmnet(X, y, alpha = 1, family = "binomial",
                lambda = cv.lasso$lambda.min)
```

```
# Display regression coefficients
coef(model)
```

```
## 8 x 1 sparse Matrix of class "dgCMatrix"
##                               s0
## (Intercept)       -3.831776e+00
## DisbursementGross -1.323195e-07
## SBA_Appv          -1.570861e-06
## UrbanRural_1       1.825798e+00
## UrbanRural_2       1.478091e+00
## RevLineCr_1       -2.636584e-01
## Below_10_years     1.246510e+00
## Recession_1        5.531605e-01
```

Now Using Elastic Net

```
# ELASTIC NET WITH 0 < ALPHA < 1
a <- seq(0.1, 0.9, 0.05)
library(foreach)
library(parallel)
search <- foreach(i = a, .combine = rbind) %dopar% {
  cv <- cv.glmnet(X, y, family = "binomial", nfold = 10, type.measure = "deviance", parallel = FALSE,
                  alpha = i)
  data.frame(cvm = cv$cvm[cv$lambda == cv$lambda.1se], lambda.1se = cv$lambda.1se, alpha = i)
}
cv3 <- search[search$cvm == min(search$cvm), ]
cv3
```

```
##          cvm  lambda.1se alpha
## 13 0.9399006 0.003691252   0.7
```

```
md3 <- glmnet(X, y, family = "binomial", lambda = cv3$lambda.1se, alpha = cv3$alpha)
coef(md3)
```

```
## 8 x 1 sparse Matrix of class "dgCMatrix"
##                               s0
## (Intercept)       -3.569397e+00
## DisbursementGross -1.879025e-07
## SBA_Appv          -1.285183e-06
## UrbanRural_1       1.591943e+00
## UrbanRural_2       1.218760e+00
## RevLineCr_1       -1.780763e-01
## Below_10_years     1.147029e+00
## Recession_1        5.434304e-01
```

Examining coefficients on the two models.

First Lasso

```
# Lasso coefficients
lasso_coef <- coef(model)

# Convert coefficients to a data frame and calculate absolute values
lasso_coef_df <- as.data.frame(as.matrix(lasso_coef))
lasso_coef_df$abs_value <- abs(lasso_coef_df$s0)

# Sort by absolute value in descending order
top_lasso <- lasso_coef_df[order(-lasso_coef_df$abs_value), ]

# Print coefficients
print(top_lasso)
```

```
##                              s0    abs_value
## (Intercept)       -3.831776e+00 3.831776e+00
## UrbanRural_1       1.825798e+00 1.825798e+00
## UrbanRural_2       1.478091e+00 1.478091e+00
## Below_10_years     1.246510e+00 1.246510e+00
## Recession_1        5.531605e-01 5.531605e-01
## RevLineCr_1       -2.636584e-01 2.636584e-01
## SBA_Appv          -1.570861e-06 1.570861e-06
## DisbursementGross -1.323195e-07 1.323195e-07
```

Then for Elastic Net.

```
# Elastic Net coefficients
elastic_net_coef <- coef(md3)

# Convert coefficients to a data frame and calculate absolute values
elastic_net_coef_df <- as.data.frame(as.matrix(elastic_net_coef))
elastic_net_coef_df$abs_value <- abs(elastic_net_coef_df$s0)

# Sort by absolute value in descending order
top_elastic_net <- elastic_net_coef_df[order(-elastic_net_coef_df$abs_value), ]

# Print the coefficients
print(top_elastic_net)
```

```
##                              s0    abs_value
## (Intercept)       -3.569397e+00 3.569397e+00
## UrbanRural_1       1.591943e+00 1.591943e+00
## UrbanRural_2       1.218760e+00 1.218760e+00
## Below_10_years     1.147029e+00 1.147029e+00
## Recession_1        5.434304e-01 5.434304e-01
```

```
## RevLineCr_1       -1.780763e-01 1.780763e-01
## SBA_Appv          -1.285183e-06 1.285183e-06
## DisbursementGross -1.879025e-07 1.879025e-07
```

Predict using Elastic Net model

```
# Predict probabilities for the validation data
valid_X_matrix <- model.matrix(~ ., data = valid_X)[, -1]

predicted_probs <- predict(md3, newx = valid_X_matrix, type = "response")

head(predicted_probs)
```

```
##          s0
## 1 0.2495627
## 2 0.2975450
## 3 0.4122066
## 4 0.2164192
## 5 0.3017746
## 6 0.4271726
```

Confusion Matrix

```
# Generate predictions (binary) for the validation data
predicted_classes <- ifelse(predicted_probs >= 0.5, 1, 0)

# Ensure valid_y contains both 0 and 1
all_classes <- c(0, 1)

# Create confusion matrix with fixed categories
confusion_matrix <- table(
  Actual = factor(valid_y$MIS_Status, levels = all_classes),
  Predicted = factor(predicted_classes, levels = all_classes)
)

# Display the confusion matrix
print(confusion_matrix)
```

```
##       Predicted
## Actual      0      1
##      0 147219      0
##      1  42193      0
```

```
# Calculate accuracy
accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)
cat("Accuracy: ", sprintf("%.2f%%", accuracy * 100), "\n")
```

```
## Accuracy:  77.72%
```

Metrics from Confusion Matrix

```
# Extract values from the confusion matrix
TN <- confusion_matrix[1, 1]  # True Negatives
FP <- confusion_matrix[1, 2]  # False Positives
FN <- confusion_matrix[2, 1]  # False Negatives
TP <- confusion_matrix[2, 2]  # True Positives

# Compute precision, recall, F1 score, and specificity with checks to avoid NaN
precision <- ifelse((TP + FP) == 0, 0, TP / (TP + FP))  # Avoid division by zero
recall <- ifelse((TP + FN) == 0, 0, TP / (TP + FN))  # Avoid division by zero
f1_score <- ifelse((precision + recall) == 0, 0, 2 * (precision * recall) / (precision + recall))  # Av
specificity <- ifelse((TN + FP) == 0, 0, TN / (TN + FP))  # Avoid division by zero

# Create a data frame to display the metrics
scores <- data.frame(
  Model = "Elastic Net Logistic Regression",
  Precision = precision,
  Recall = recall,
  `F1 Score` = f1_score,
  Specificity = specificity
)

# Display the scores
print(scores)
```

```
##                             Model Precision Recall F1.Score Specificity
## 1 Elastic Net Logistic Regression         0      0        0           1
```

Gains and Lift Charts

```
# Install and load required package
if (!require(gains)) install.packages("gains", dependencies = TRUE)
library(gains)

# Example data setup (replace with your actual data)
set.seed(123)
valid_y <- data.frame(MIS_Status = sample(0:1, 100, replace = TRUE))  # Actual labels
predicted_probs <- runif(100)  # Random probabilities for demonstration

# Create a results data frame
logit_result <- data.frame(
  actual = valid_y$MIS_Status,
  p_0 = 1 - predicted_probs,
  p_1 = predicted_probs,
  predicted = ifelse(predicted_probs > 0.5, 1, 0)
)

# Sort the validation dataset by predicted probabilities (highest first)
logit_result <- logit_result[order(-logit_result$p_1), ]
```

```r
# Create gains chart
gains_chart <- gains(
  actual = logit_result$actual,
  predicted = logit_result$p_1
)

# Baseline (random) model cumulative percentages
baseline_percentage = seq(10, 100, by = 10)  # 10%, 20%, ..., 100%
baseline_cum_pct = baseline_percentage / 100  # Cumulative percentage for random model

# Model's cumulative percentage (cumulative sum of actual positives in each decile)
model_cum_pct = gains_chart$cume.pct.of.total / 100  # Model's cumulative percentage of positives

# Corrected Lift Calculation
# Lift for each decile: Model's cumulative percentage / Random cumulative percentage
actual_lift = model_cum_pct / baseline_cum_pct

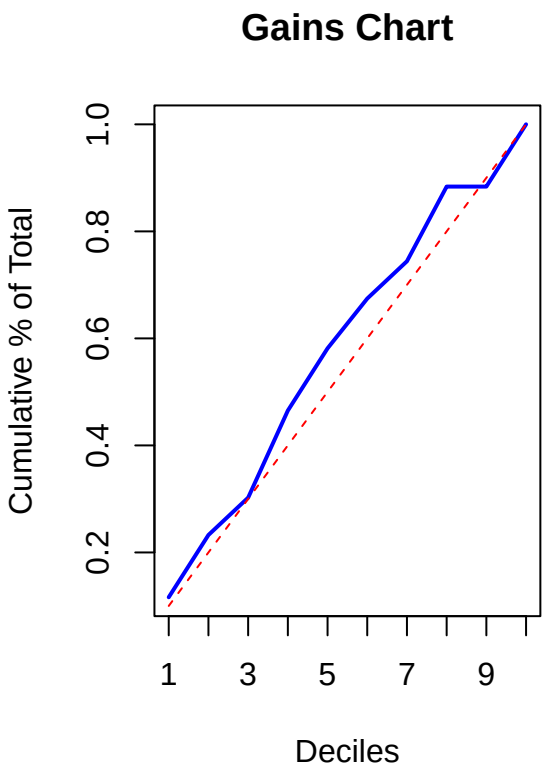
# Multiply the lift by 100 to convert it to percentage
actual_lift_percentage = actual_lift * 100

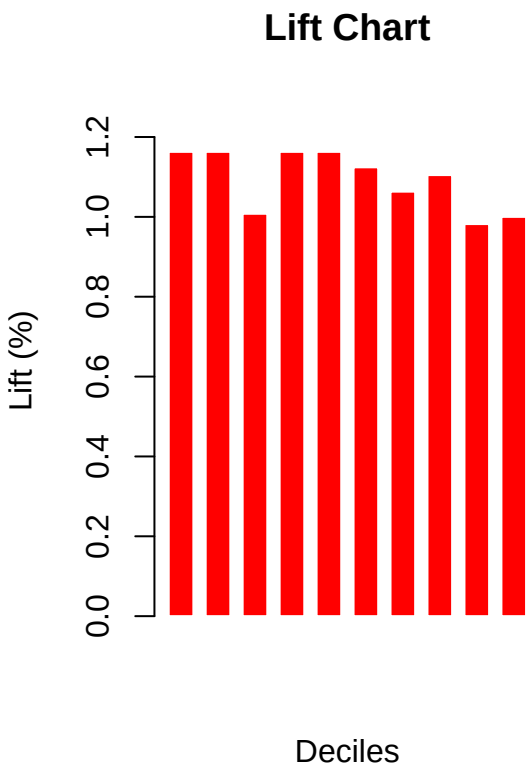
# Set up side-by-side plotting
par(mfrow = c(1, 2))  # 1 row, 2 columns

# Gains Chart (Line Plot)
plot(
  seq_along(gains_chart$cume.pct.of.total),  # Deciles as sequence
  gains_chart$cume.pct.of.total,
  type = "l", col = "blue", lwd = 2,
  xlab = "Deciles", ylab = "Cumulative % of Total",
  main = "Gains Chart",
  xaxt = "n"
)
axis(1, at = seq_along(gains_chart$cume.pct.of.total), labels = gains_chart$deciles)

# Add the diagonal dotted line representing the random model (baseline)
lines(seq_along(baseline_cum_pct), baseline_cum_pct * max(gains_chart$cume.pct.of.total), col = "red", l

# Lift Chart (Bar Chart with Corrected Lift Values in Percentage)
barplot(
  actual_lift_percentage,
  names.arg = gains_chart$deciles,
  col = "red",
  border = "white",
  main = "Lift Chart",
  xlab = "Deciles",
  ylab = "Lift (%)",
  space = 0.5,  # Adjust spacing between bars
  ylim = c(0, max(actual_lift_percentage) * 1.1)  # Set y-axis limit slightly above max lift
)
```






```
# Reset plotting layout
par(mfrow = c(1, 1))
```

Net Profit

```
# Assuming you have your glmnet model as 'model' and 'train_X' as the training data

# Convert train_X to a matrix, as glmnet expects a matrix of predictors
train_X_matrix <- as.matrix(train_X)

# Get predicted probabilities for the positive class (1: "Default")
pred_prob <- predict(model, newx = train_X_matrix, type = "response", s = 0.1)  # Use s = 0.1 or anothe

# Initialize an array to store net profits for each threshold
thresholds <- seq(0.01, 0.99, length.out = 100)  # Range of thresholds from 0.01 to 0.99
net_profits <- numeric(length(thresholds))  # Empty vector to store net profits

# Convert actual_y to numeric
disbursement_gross <- train_X$DisbursementGross
actual_y <- train_y

# Loop over each threshold and calculate the net profit
for (i in 1:length(thresholds)) {
```

```r
  threshold <- thresholds[i]

  # Classify based on the threshold
  pred_y <- as.integer(pred_prob >= threshold)  # 1 if predicted probability >= threshold, else 0

  # Initialize net_profit vector for this threshold
  net_profit <- numeric(length(actual_y))

  # Case 1: Correctly classified as "Paid in full" (Actual = 0, Predicted = 0)
  mask <- (actual_y == 0) & (pred_y == 0)
  net_profit[mask] <- 0.05 * disbursement_gross[mask]

  # Case 2: Incorrectly classified as "Paid in full" (Actual = 1, Predicted = 0)
  mask <- (actual_y == 1) & (pred_y == 0)
  net_profit[mask] <- -0.05 * disbursement_gross[mask] * 5

  # Case 3: Correctly classified as "Default" (Actual = 1, Predicted = 1)
  mask <- (actual_y == 1) & (pred_y == 1)
  net_profit[mask] <- 0

  # Case 4: Incorrectly classified as "Default" (Actual = 0, Predicted = 1)
  mask <- (actual_y == 0) & (pred_y == 1)
  net_profit[mask] <- 0  # No profit or loss for incorrect "default"

  # Calculate total net profit for this threshold
  net_profits[i] <- sum(net_profit)
}

# Find the threshold that maximizes the net profit
best_threshold <- thresholds[which.max(net_profits)]
max_net_profit <- max(net_profits)

# Output the best threshold and corresponding net profit
cat(sprintf("Best Threshold: %.2f\n", best_threshold))
```

```
## Best Threshold: 0.24
```

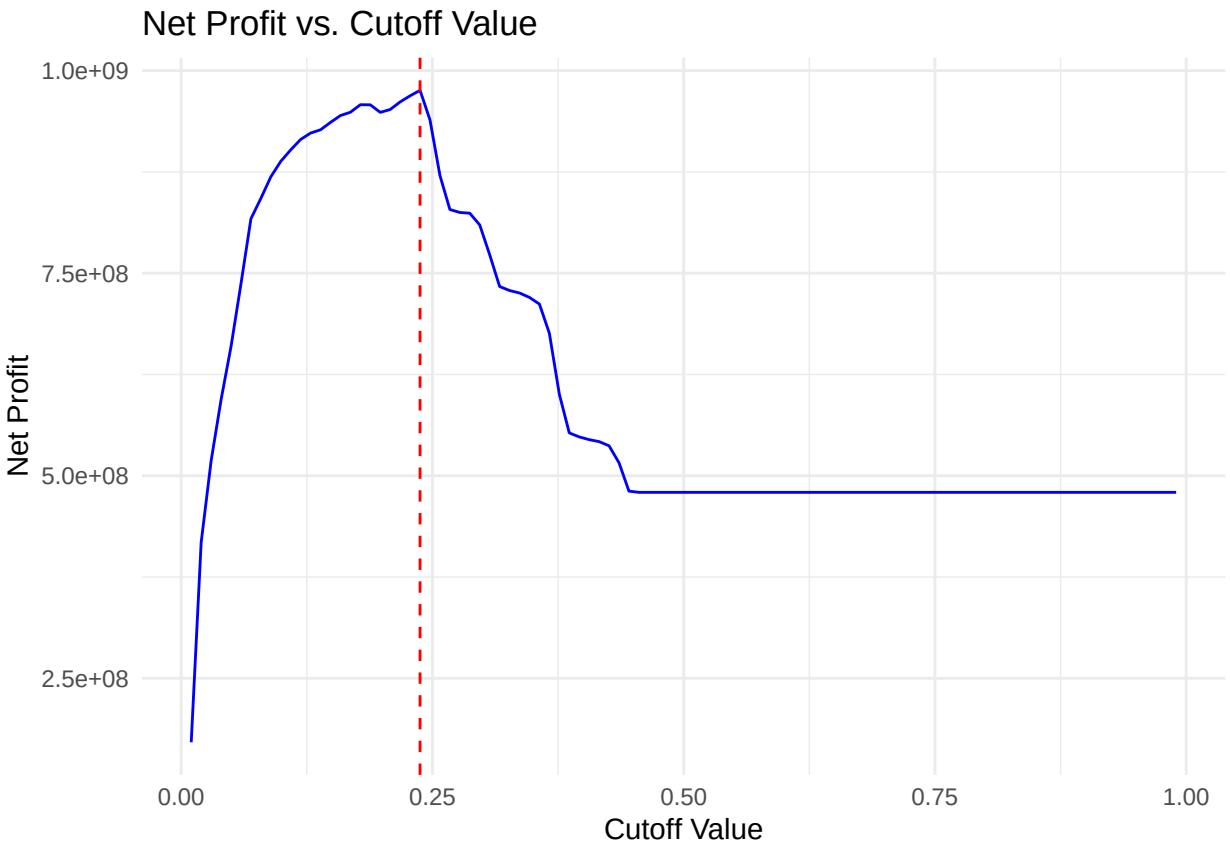
```r
cat(sprintf("Maximum Net Profit: %.2f USD\n", max_net_profit))
```

```
## Maximum Net Profit: 975679101.55 USD
```

```r
# Plot the net profit vs. threshold
library(ggplot2)

profit_df <- data.frame(cutoff = thresholds, profit = net_profits)

ggplot(profit_df, aes(x = cutoff, y = profit)) +
  geom_line(color = "blue") +
  geom_vline(xintercept = best_threshold, linetype = "dashed", color = "red") +
  labs(title = "Net Profit vs. Cutoff Value",
       x = "Cutoff Value",
       y = "Net Profit") +
  theme_minimal()
```



Given that Elastic Net is a mix of Lasso and Ridge,it is not surprising that it has a better performance than Lasso.